**Université Iba Der Thiam de Thiès**

UFR Sciences et Technologies

Département Informatique

**Master 2 – Génie Logiciel — 2025 – 2026**

# Simulation de Circulation en Java Concurrente

*Carrefour • Tunnel • Parking*

---

Rapport de projet – Systèmes Distribués Multi-Tiers

Gestion de la concurrence avec Java `java.util.concurrent`

**Fatou FALL**

**Sophie FALL**

**Cheikhouna GUEYE**

Année académique 2025 – 2026

# Contents

**Résumé** **4**

**1 Introduction et Contexte** **5**
1.1 Contexte académique . . . . . 5
1.2 Problématique . . . . . 5
1.3 Objectifs du projet . . . . . 6
1.4 Technologies utilisées . . . . . 7

**2 Architecture du Projet** **8**
2.1 Structure des packages . . . . . 8
2.2 Description des composants . . . . . 9
2.2.1 Package `common` . . . . . 9
2.2.2 Packages de simulation . . . . . 9
2.2.3 Package `ui` . . . . . 9
2.3 Synthèse des mécanismes de synchronisation . . . . . 10

**3 Simulation du Carrefour** **11**
3.1 Description du problème . . . . . 11
3.2 Règle de priorité à droite . . . . . 12
3.3 Détection du blocage circulaire . . . . . 12
3.4 Implémentation Java . . . . . 13
3.4.1 Classe `Carrefour` . . . . . 13
3.4.2 Points clés de l'implémentation . . . . . 14
3.5 Démonstration . . . . . 15
3.5.1 Journal d'exécution commenté . . . . . 15

**4 Simulation du Tunnel** **16**
4.1 Description du problème . . . . . 16

4.2 Modèle de synchronisation . . . . . 16
4.3 Implémentation Java . . . . . 17
4.4 Propriétés garanties . . . . . 18
4.5 Démonstration . . . . . 19

**5 Simulation du Parking** **21**
5.1 Description du problème . . . . . 21
5.2 Architecture de synchronisation . . . . . 22
5.3 Implémentation Java . . . . . 22
5.4 Scénario de test . . . . . 23

**6 Interface Graphique** **25**
6.1 Vue d'ensemble . . . . . 25
6.2 Organisation de la fenêtre . . . . . 26
6.3 Composants personnalisés . . . . . 26
6.4 Journal coloré des événements . . . . . 26
6.5 Thread-safety de l'interface . . . . . 27

**7 Analyse et Discussion** **28**
7.1 Comparaison des mécanismes . . . . . 28
7.2 Gestion du deadlock . . . . . 29
7.3 Limitations et perspectives . . . . . 29
7.3.1 Limitations actuelles . . . . . 29
7.3.2 Perspectives d'amélioration . . . . . 29

**Conclusion** **30**

**Annexe – Guide d'exécution** **31**

# List of Figures


2.1 Structure des packages du projet . . . . . 8

3.1 Schéma du carrefour à priorité à droite (sujet de cours, 24/03/2024) . . 11
3.2 Interface lors de la simulation du carrefour (6 voitures) . . . . . 15

4.1 Schéma du tunnel à une voie (sujet de cours) . . . . . 16
4.2 Interface lors de la simulation du tunnel (6 voitures) . . . . . 19

5.1 Schéma du parking à $N$ places (sujet d'examen, 09/03/2024) . . . . . 21
5.2 Interface lors de la simulation du parking (5 voitures, 2 places, 2 portes) 23

6.1 Fenêtre principale de l'application au démarrage . . . . . 25

# Résumé

Ce rapport présente un projet de simulation de circulation routière développé en Java concurrente dans le cadre du cours de **Systèmes Distribués Multi-Tiers** (Master 2 – Génie Logiciel, Université Iba Der Thiam de Thiès).

Le projet illustre trois scénarios classiques de gestion de ressources partagées entre threads concurrents :

- **Un carrefour avec priorité à droite** : gestion de l'accès exclusif à une intersection avec détection de blocage circulaire (deadlock), implémentée via `ReentrantLock` et `Condition`.
- **Un tunnel à une voie** : contrôle de flux directionnel exclusif, où un seul sens de circulation est autorisé à la fois.
- **Un parking à capacité limitée** : gestion de places et de portes multiples via des `Semaphore`s, avec comptage atomique des véhicules présents.

Chaque scénario est implémenté avec les primitives de synchronisation du package `java.util.concurrent`, et l'ensemble est piloté depuis une interface graphique interactive développée en Java Swing. L'interface affiche en temps réel un journal coloré des événements de chaque simulation.

**Mots-clés :** Java 17, Threads, ReentrantLock, Condition, Semaphore, AtomicInteger, Java Swing, Concurrence, Synchronisation, Ressources partagées, Systèmes distribués.

# Chapter 1

# Introduction et Contexte

## 1.1 Contexte académique

Ce projet s'inscrit dans le cadre du module **Systèmes Distribués Multi-Tiers** dispensé en Master 2 Génie Logiciel à l'Université Iba Der Thiam de Thiès. Ce module aborde notamment les problématiques de concurrence, de synchronisation et de gestion des ressources partagées entre processus ou threads s'exécutant en parallèle.

Les exercices proposés en cours portaient sur trois situations de la vie courante modélisant ces problèmes de manière concrète :

1. Le carrefour avec règle de priorité à droite (exercice du 24/03/2024)
2. Le tunnel à une voie bidirectionnelle
3. Le parking à $N$ places avec une ou plusieurs portes (examen du 09/03/2024)

## 1.2 Problématique

Dans tout système concurrent, plusieurs entités indépendantes tentent d'accéder simultanément à des ressources partagées. Sans mécanismes de synchronisation appropriés, ces accès concurrents peuvent provoquer :

- Des **conditions de course** (*race conditions*) : résultats imprévisibles selon l'ordre d'exécution des threads.
- Des **interblocages** (*deadlocks*) : situation où deux ou plusieurs threads s'attendent mutuellement, bloquant indéfiniment l'exécution.
- De la **famine** (*starvation*) : un thread n'obtient jamais accès à la ressource car d'autres le précèdent constamment.

**Exemple concret :** dans le scénario du carrefour, si quatre voitures arrivent simultanément de chacune des quatre directions, chacune attend la priorité de son voisin de droite. Sans mécanisme de détection, cette situation constitue un deadlock circulaire parfait.

## 1.3 Objectifs du projet

---

**Objectifs principaux**

1. **Modéliser** trois scénarios de circulation en termes de threads et de ressources partagées.
2. **Implémenter** les mécanismes de synchronisation adaptés à chaque scénario.
3. **Détecter et résoudre** les situations de blocage circulaire.
4. **Visualiser** le comportement concurrent en temps réel via une interface graphique.

## 1.4 Technologies utilisées

**Table 1.1.** Technologies et bibliothèques utilisées

| **Technologie** | Rôle dans le projet |
|---|---|
| **Java 17** | Langage de développement principal |
| `java.util.concurrent` | Primitives de synchronisation (Lock, Condition, Semaphore) |
| `java.util.concurrent.atomic` | Compteurs thread-safe (AtomicInteger) |
| `javax.swing` | Interface graphique interactive |
| `java.lang.Thread` | Modélisation des voitures comme threads |
| **IntelliJ IDEA Community** | Environnement de développement |

# Chapter 2

# Architecture du Projet

## 2.1 Structure des packages

Le projet adopte une architecture modulaire organisée en cinq packages distincts. Cette organisation respecte le principe de *séparation des responsabilités* (*Separation of Concerns*), chaque package encapsulant la logique d'un composant précis.

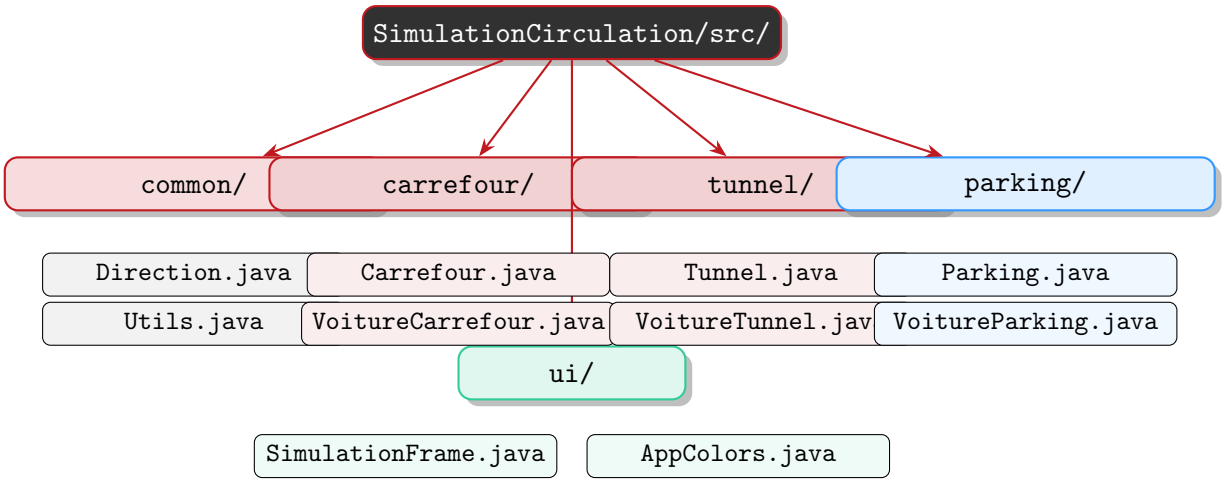



**Figure 2.1.** Structure des packages du projet

## 2.2 Description des composants

### 2.2.1 Package `common`

Ce package fournit les utilitaires partagés par tous les autres modules :

- `Direction` : énumération définissant les directions de déplacement (`A`, `B`, `C`, `D` pour le carrefour ; `GAUCHE_DROITE` et `DROITE_GAUCHE` pour le tunnel).
- `Utils` : classe utilitaire fournissant un logger horodaté avec injection de consommateur (`Consumer<String>`) pour rediriger les logs vers l'interface graphique.

### 2.2.2 Packages de simulation

Chaque scénario est encapsulé dans son propre package, suivant un patron commun :

- Une classe **ressource partagée** (ex. `Carrefour`) gérant la synchronisation.
- Une classe **thread** héritant de `Thread` (ex. `VoitureCarrefour`) représentant un véhicule.

### 2.2.3 Package `ui`

L'interface graphique est entièrement contenue dans ce package :

- `SimulationFrame` : fenêtre principale gérant les paramètres, le lancement des simulations et l'affichage des logs.
- `AppColors` : palette de couleurs centralisée.
- `RoundedButton`, `RoundedPanel` : composants Swing personnalisés avec rendu anti-aliasé.

## 2.3 Synthèse des mécanismes de synchronisation

**Table 2.1.** Mécanismes de synchronisation par scénario

| **Scénario** | Classe ressource | Mécanisme | Comportement |
|---|---|---|---|
| **Carrefour** | `Carrefour` | `ReentrantLock(true)` + `Condition` | Priorité à droite, détection deadlock |
| **Tunnel** | `Tunnel` | `ReentrantLock(true)` + `Condition` | Sens unique, file par direction |
| **Parking** | `Parking` | `Semaphore` (places + portes) | Capacité limitée, portes mutex |
| **UI** | `SimulationFrame` | `SwingUtilities.invokeLater` | Mises à jour UI thread-safe |

# Chapter 3

# Simulation du Carrefour

## 3.1 Description du problème

Le carrefour simule une intersection à quatre branches, étiquetées A (nord), B (ouest), C (sud) et D (est). La règle de circulation appliquée est la **priorité à droite** : une voiture ne peut entrer dans le carrefour que si aucune voiture n'attend à sa droite immédiate.

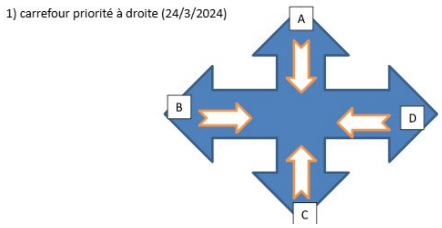
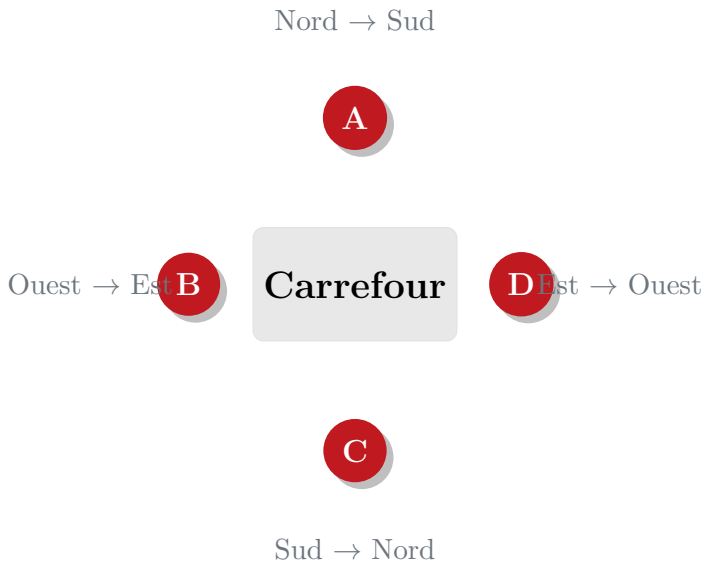



**Figure 3.1.** Schéma du carrefour à priorité à droite (sujet de cours, 24/03/2024)

La correspondance des directions est la suivante :




## 3.2 Règle de priorité à droite

Pour chaque direction, la direction ayant la priorité est définie par la fonction `droite(d)` :

$$A \xrightarrow{droite} B \xrightarrow{droite} C \xrightarrow{droite} D \xrightarrow{droite} A$$

Une voiture venant de la direction $d$ doit attendre si la direction droite($d$) a des véhicules en attente.

## 3.3 Détection du blocage circulaire

**Cas de deadlock :** si les quatre directions A, B, C et D ont simultanément des voitures en attente, chacune attend la priorité de son voisin de droite. Ce cycle d'attente circulaire constitue un deadlock parfait.

La méthode `toutLeMondeAttend()` détecte cette situation en vérifiant que chaque direction a au moins un véhicule en attente. Quand ce cas est détecté, la contrainte de priorité est levée temporairement pour permettre à la première voiture de passer.

## 3.4 Implémentation Java

### 3.4.1 Classe Carrefour

```java
public void entrer(Direction direction, int voitureId)
        throws InterruptedException {
    lock.lock();
    try {
        attente.put(direction, attente.get(direction) + 1);
        Utils.log("[INFO] La voiture " + voitureId +
                  " arrive au carrefour depuis la direction
                     " + direction);

        // Attendre si priorite a droite OU si carrefour
           occupe
        while (occupe ||
               (attente.get(droite(direction)) > 0
                && !toutLeMondeAttend())) {
            condition.await();
        }

        // Deblocage du deadlock circulaire
        if (!occupe && toutLeMondeAttend()) {
            Utils.log("[DEBLOCAGE] Blocage circulaire
               detecte. "
                     + "La voiture " + voitureId + " est
                        autorisee.");
        }

        attente.put(direction, attente.get(direction) - 1);
        occupe = true;
    } finally {
        lock.unlock();
    }
}

public void sortir(Direction direction, int voitureId) {
    lock.lock();
    try {
```

```
32            occupe = false;
33            condition.signalAll(); // Notifier tous les threads
                 en attente
34        } finally {
35            lock.unlock();
36        }
37    }
```

**Listing 3.1.** Méthode `entrer()` de la classe Carrefour

### 3.4.2 Points clés de l'implémentation

- `ReentrantLock(true)` : le paramètre `true` active le mode *fair* (équitable). Les threads acquièrent le verrou dans l'ordre FIFO, éliminant le risque de famine.
- `condition.await()` : libère le verrou et suspend le thread jusqu'à être notifié.
- `condition.signalAll()` : réveille tous les threads en attente à chaque sortie.
- La boucle `while` (et non `if`) protège contre les réveils spurieux (*spurious wake-ups*).
- L'`EnumMap` `attente` mémorise, pour chaque direction, le nombre de voitures en attente.

## 3.5 Démonstration

**Figure 3.2.** Interface lors de la simulation du carrefour (6 voitures)

### 3.5.1 Journal d'exécution commenté

**Extrait du journal – Carrefour (6 voitures)**

```
[INFO]  La voiture 1 arrive au carrefour depuis la direction D.
[PASSAGE]  La voiture 1 entre dans le carrefour depuis la direction D.
[INFO]  La voiture 2 arrive au carrefour depuis la direction A.
[ATTENTE]  La voiture 2 attend depuis A -- priorité à B.
[INFO]  La voiture 4 arrive au carrefour depuis la direction B.
[ATTENTE]  La voiture 4 attend depuis B -- priorité à C.
[SORTIE]  La voiture 1 a quitté le carrefour depuis la direction D.
[PASSAGE]  La voiture 4 entre dans le carrefour depuis la direction B.
[SORTIE]  La voiture 4 a quitté le carrefour depuis la direction B.
[PASSAGE]  La voiture 5 entre dans le carrefour depuis la direction B.
[PASSAGE]  La voiture 2 entre dans le carrefour depuis la direction A.
[FIN]  La simulation du carrefour est terminée.
```

**Analyse :** la voiture 1 (direction D) entre immédiatement car aucune voiture n'attend à sa droite (direction A). La voiture 2 (direction A) attend car B a la priorité. Ce comportement valide bien la règle de priorité à droite.

# Chapter 4

# Simulation du Tunnel

## 4.1 Description du problème

Le tunnel simule une voie unique bidirectionnelle. Une seule direction de circulation est autorisée à la fois : `GAUCHE_DROITE` ou `DROITE_GAUCHE`. Les voitures voyageant dans le sens opposé doivent attendre que le tunnel soit complètement vide avant de pouvoir entrer.

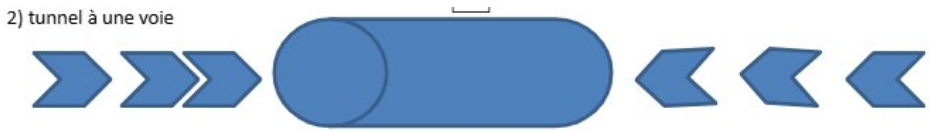



**Figure 4.1.** Schéma du tunnel à une voie (sujet de cours)

## 4.2 Modèle de synchronisation

**Variables d'état du tunnel**

- `sensCourant` : direction actuellement autorisée dans le tunnel (`null` si vide).
- `nbDansTunnel` : compteur du nombre de voitures actuellement dans le tunnel.

La règle d'entrée est la suivante : une voiture ne peut entrer que si `sensCourant == null` (tunnel vide) ou `sensCourant == sens` (même direction qu'elle). Sinon, elle attend via `condition.await()`.

## 4.3 Implémentation Java

```java
public void entrer(Direction sens, int voitureId)
        throws InterruptedException {
    lock.lock();
    try {
        // Attendre si le tunnel est occupe dans le sens oppose
        while (sensCourant != null && sensCourant != sens) {
            Utils.log("[ATTENTE] La voiture " + voitureId +
                      " patiente (sens oppose dans le tunnel).");
            condition.await();
        }
        sensCourant = sens;
        nbDansTunnel++;
        Utils.log("[PASSAGE] La voiture " + voitureId +
                  " entre dans le tunnel. Sens : " + sens +
                  ". Voitures : " + nbDansTunnel);
    } finally {
        lock.unlock();
    }
}

public void sortir(Direction sens, int voitureId) {
    lock.lock();
    try {
        nbDansTunnel--;
        if (nbDansTunnel == 0) {
            sensCourant = null; // Tunnel vide : changement de sens possible
            Utils.log("[INFO] Tunnel vide. Changement de sens possible.");
            condition.signalAll();
        }
```

```
30      } finally {
31          lock.unlock();
32      }
33  }
```

**Listing 4.1.** Méthodes `entrer()` et `sortir()` de la classe Tunnel

## 4.4 Propriétés garanties

**Table 4.1.** Propriétés de la simulation du tunnel

| **Propriété** | Garantie par |
|---|---|
| **Exclusion mutuelle des sens** | `while (sensCourant != null && sensCourant != sens)` |
| **Pas de famine** | `ReentrantLock(true)` – mode équitable (FIFO) |
| **Notification correcte** | `signalAll()` uniquement quand le tunnel est vide |
| **Cohérence du compteur** | Incrémentation/décrémentation sous verrou |

## 4.5 Démonstration

**Figure 4.2.** Interface lors de la simulation du tunnel (6 voitures)

**Extrait du journal – Tunnel (6 voitures)**

```
[INFO] La voiture 1 souhaite entrer dans le tunnel en direction
DROITE_GAUCHE.
[PASSAGE] La voiture 1 entre dans le tunnel. Sens : DROITE_GAUCHE.
Voitures : 1.
[ATTENTE] La voiture 2 patiente (tunnel occupé sens opposé).
[ATTENTE] La voiture 3 patiente (tunnel occupé sens opposé).
[PASSAGE] La voiture 6 entre dans le tunnel. Sens : DROITE_GAUCHE.
Voitures : 2.
[SORTIE] La voiture 1 quitte le tunnel. Voitures restantes : 1.
[SORTIE] La voiture 6 quitte le tunnel. Voitures restantes : 0.
[INFO] Le tunnel est maintenant vide. Un changement de sens est
possible.
[PASSAGE] La voiture 2 entre dans le tunnel. Sens : GAUCHE_DROITE.
Voitures : 1.
[PASSAGE] La voiture 3 entre dans le tunnel. Voitures : 2.
[PASSAGE] La voiture 4 entre dans le tunnel. Voitures : 3.
[PASSAGE] La voiture 5 entre dans le tunnel. Voitures : 4.
[FIN] La simulation du tunnel est terminée.
```

**Analyse :** les voitures 2, 3, 4, 5 (direction `GAUCHE_DROITE`) sont bloquées tant que les

voitures 1 et 6 (`DROITE_GAUCHE`) n'ont pas quitté le tunnel. Dès que le tunnel se vide, `signalAll()` libère toutes les voitures en attente qui entrent simultanément dans le même sens.

# Chapter 5

# Simulation du Parking

## 5.1 Description du problème

Le parking modélise un espace de stationnement à $N$ places et $P$ portes. Chaque voiture doit :

1. **Obtenir une place** libre (si le parking est plein, elle attend).
2. **Accéder à une porte** d'entrée (accès exclusif par porte).
3. **Stationner** pendant une durée aléatoire.
4. **Quitter** via une porte de sortie (là encore, accès exclusif).

3) Parking de n places. Toute voiture qui entre doit avoir une place. (Examen 9/3/2024)

~~a) une porte~~

b) plusieurs portes

Toute voiture qui entre doit trouver une place

**Figure 5.1.** Schéma du parking à $N$ places (sujet d'examen, 09/03/2024)

## 5.2 Architecture de synchronisation

Deux niveaux de sémaphores sont utilisés :

**Structure des sémaphores**

- `Semaphore places` ($N$) : contrôle la capacité globale. `acquire()` bloque si toutes les places sont occupées ; `release()` libère une place à la sortie.
- `Semaphore[] portes` (1 par porte) : mutex sur chaque porte. Un seul véhicule peut franchir une porte à la fois (sémaphore binaire initialisé à 1).

Un `AtomicInteger voituresPresentes` permet d'afficher l'occupation courante de manière thread-safe, sans verrou supplémentaire.

## 5.3 Implémentation Java

```java
public void entrer(int voitureId, int porte)
        throws InterruptedException {
    int indexPorte = porte - 1;

    if (places.availablePermits() == 0) {
        Utils.log("[ATTENTE] La voiture " + voitureId +
                  " patiente : parking complet.");
    }
    places.acquire(); // Attendre une place libre

    Utils.log("[INFO] La voiture " + voitureId +
              " a une place, attend l'acces a la porte " +
                porte);

    portes[indexPorte].acquire(); // Acces exclusif a la
        porte
    try {
        Utils.log("[PASSAGE] La voiture " + voitureId +
                  " franchit la porte d'entree " + porte);
        Utils.sleep(1000); // Traversee de la porte
        int presentes = voituresPresentes.incrementAndGet();
        Utils.log("[PASSAGE] La voiture " + voitureId +
                  " est entree. Occupation : " + presentes +
                    "/" + capaciteMax);
```

```
22      } finally {
23          portes[indexPorte].release(); // Liberer la porte
                apres traversee
24      }
25  }
```

**Listing 5.1.** Méthode `entrer()` de la classe Parking

## 5.4 Scénario de test

Le test a été réalisé avec la configuration suivante : **5 voitures**, **capacité 2 places**, **2 portes**.

**Figure 5.2.** Interface lors de la simulation du parking (5 voitures, 2 places, 2 portes)

**Extrait du journal – Parking (5 voitures, 2 places, 2 portes)**

```
[INFO]  La voiture 1 demande l'accès par la porte 2.
[PASSAGE]  La voiture 1 franchit la porte d'entrée 2.
[INFO]  La voiture 2 obtient une place -- attend la porte 2.
[ATTENTE]  La voiture 3 patiente :  parking complet (2/2).
[ATTENTE]  La voiture 4 patiente :  parking complet.
[PASSAGE]  La voiture 2 est entrée.  Occupation :  2/2.
[SORTIE]  La voiture 1 quitte par la porte 2.  Occupation :  1/2.
[PASSAGE]  La voiture 3 franchit la porte d'entrée 1.  Occupation :
```

```
2/2.
[SORTIE]  La voiture 5 quitte par la porte 2.  Occupation :  0/2.
[FIN]  La simulation du parking est terminée.
```

**Analyse :** les voitures 3, 4, 5 sont bloquées dès que le parking est plein (2/2). Chaque sortie libère un permis du sémaphore `places`, permettant à la prochaine voiture en attente d'entrer. On observe également que deux voitures peuvent être en transit simultanément si elles utilisent des portes différentes.

# Chapter 6

# Interface Graphique

## 6.1 Vue d'ensemble

L'interface graphique est développée en Java Swing. Elle permet de configurer et lancer les trois simulations depuis une fenêtre unique, et affiche en temps réel un journal coloré des événements.

**Figure 6.1.** Fenêtre principale de l'application au démarrage

## 6.2 Organisation de la fenêtre

La fenêtre est structurée en trois zones principales :

- **En-tête** : titre du projet avec fond coloré (`RoundedPanel`).
- **Panneau gauche** – Paramètres : menu de sélection du type de simulation, champs de configuration (nombre de voitures, capacité, portes), et boutons d'action.
- **Panneau droit** – Journal : zone de texte stylisée (`JTextPane`) affichant les logs avec couleurs différenciées par catégorie.

## 6.3 Composants personnalisés

`RoundedButton` Bouton avec coins arrondis et effet hover, dessiné via `Graphics2D` avec anti-aliasing.

`RoundedPanel` Panneau avec fond arrondi, base de toutes les cartes de l'interface.

`AppColors` Centralise la palette de couleurs : `PRIMARY`, `ACCENT`, `SUCCESS`, `DANGER`, etc.

## 6.4 Journal coloré des événements

**Table 6.1.** Code couleur du journal des événements

| Tag | Couleur | Signification |
|---|---|---|
| `[INFO]` | **Bleu** | Arrivée d'un véhicule, état système |
| `[ATTENTE]` | **Orange** | Véhicule bloqué, en attente |
| `[PASSAGE]` | **Vert** | Véhicule en cours de traversée |
| `[SORTIE]` | **Rouge** | Véhicule ayant quitté la ressource |
| `[DÉBLOCAGE]` | **Violet** | Résolution d'un deadlock |
| `[FIN]` | **Gris** | Fin de simulation |

## 6.5 Thread-safety de l'interface

Java Swing n'est pas thread-safe : les mises à jour de l'UI doivent être effectuées depuis le thread de dispatch des événements (*Event Dispatch Thread*, EDT). Toutes les mises à jour de l'interface sont donc encapsulées via :

```
SwingUtilities.invokeLater(() -> {
    // Toute modification du JTextPane, JLabel, etc.
    statusLabel.setText(text);
    doc.insertString(doc.getLength(), message + "\n", style);
});
```

**Listing 6.1.** Mise à jour thread-safe de l'interface Swing

Les simulations s'exécutent dans des threads séparés (via `new Thread(() -> {...}).start()`) pour ne pas bloquer l'EDT et garder l'interface réactive pendant l'exécution.

# Chapter 7

# Analyse et Discussion

## 7.1 Comparaison des mécanismes

Table 7.1. Comparaison ReentrantLock vs Semaphore

| Critère | ReentrantLock + Condition | Semaphore |
|---|---|---|
| **Usage principal** | Exclusion mutuelle avec conditions complexes | Contrôle d'accès à $N$ ressources |
| **Flexibilité** | Conditions multiples, logique arbitraire | Simple compteur de permis |
| **Équité** | Configurable (`fair=true`) | Configurable (`fair=true`) |
| **Réentrée** | Oui (même thread peut reacquérir) | Non |
| **Notification** | `signal()` / `signalAll()` ciblé | Automatique à `release()` |
| **Adapté pour** | Carrefour, Tunnel (logique conditionnelle) | Parking (ressource quantitative) |

## 7.2 Gestion du deadlock

Le deadlock circulaire du carrefour est un cas classique du *problème des philosophes* adapté à la circulation. Notre solution consiste à détecter la situation via `toutLeMondeAttend()` et à lever temporairement la contrainte. Cette approche correspond à la stratégie de **détection + résolution** (par opposition à la prévention stricte).

**Avantage :** la solution préserve la règle de priorité dans 99% des cas tout en garantissant la *vivacité* (*liveness*) du système – aucun thread ne peut être bloqué indéfiniment.

## 7.3 Limitations et perspectives

### 7.3.1 Limitations actuelles

- La simulation du tunnel peut souffrir de **famine** dans des cas extrêmes : si un flux continu de voitures arrive dans un sens, les voitures du sens opposé peuvent attendre très longtemps (bien que le mode *fair* atténue ce risque).
- L'interface ne fournit pas de visualisation graphique animée – seul le journal texte est disponible.
- Les paramètres de timing (durées de traversée) sont fixes ou pseudo-aléatoires.

### 7.3.2 Perspectives d'amélioration

1. **Animation 2D** : visualisation graphique des véhicules se déplaçant sur le carrefour/tunnel/parking.
2. **Statistiques** : compteurs de throughput, temps d'attente moyen, taux d'occupation.
3. **Réseau de carrefours** : simulation distribuée de plusieurs intersections interconnectées.
4. **Export des logs** : sauvegarde en CSV/JSON pour analyse ultérieure.
5. **Mode pas-à-pas** : exécution contrôlée manuellement pour l'analyse pédagogique.
6. **ReadWriteLock pour le tunnel** : permettre à plusieurs voitures du même sens de traverser en lecture, tout en bloquant l'écriture (changement de sens).

# Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux des systèmes distribués et de la programmation concurrente, à travers trois scénarios concrets et progressivement complexes.

Les principaux acquis sont les suivants :

- **Maîtrise des primitives de synchronisation Java** : `ReentrantLock`, `Condition`, `Semaphore`, `AtomicInteger` – chaque outil choisi en fonction de la nature du problème.
- **Gestion des situations critiques** : détection et résolution du deadlock circulaire du carrefour, évitement de la famine grâce au mode équitable des verrous.
- **Architecture modulaire** : séparation claire entre logique métier (synchronisation) et couche présentation (Swing), facilitant la maintenance et l'évolutivité.
- **Interface réactive** : utilisation correcte de l'EDT Swing pour garantir une interface graphique fluide malgré l'exécution concurrente des simulations.

Ce projet illustre concrètement comment les problèmes de concurrence que l'on rencontre dans les systèmes distribués multi-tiers – accès concurrent à des bases de données, gestion de sessions, équilibrage de charge – peuvent être modélisés et résolus avec les outils fournis par la bibliothèque standard Java.

**Bilan :** 3 scénarios simulés, 4 mécanismes de synchronisation distincts, une interface graphique interactive, et un système 100% thread-safe par conception.

# Annexe – Guide d'exécution

## Prérequis

| Élément | Version requise |
|---|---|
| Java JDK | 17 ou supérieur |
| IntelliJ IDEA | Community Edition (gratuite) |

## Compilation et exécution

**Méthode 1 – Via IntelliJ IDEA :**

1. Ouvrir le projet dans IntelliJ IDEA Community.
2. Localiser `src/Main.java`.
3. Clic droit → *Run 'Main.main()'*.

**Méthode 2 – Via terminal (Linux/Mac) :**

```
# Se placer dans le dossier racine du projet
cd SimulationCirculation

# Compiler
javac -d out src/Main.java src/carrefour/*.java \
      src/common/*.java src/parking/*.java \
```

```
      src/tunnel/*.java src/ui/*.java

# Executer
java -cp out Main
```

**Listing 7.1.** Compilation et exécution en ligne de commande

# Utilisation de l'interface

---

1. Sélectionner le type de simulation dans le menu déroulant.
2. Saisir le nombre de voitures (défaut : 6).
3. Pour le parking uniquement : renseigner la capacité et le nombre de portes.
4. Cliquer sur **Lancer** pour démarrer la simulation.
5. Observer le journal coloré dans le panneau droit.
6. Cliquer sur **Effacer** pour vider le journal entre deux simulations.